

```
//SIT 55 : ASSIGNMENT 5 : BINARY SEARCH TREE
/*Implement binary search tree and perform following operations:
a) Insert (Handle insertion of duplicate entry)
b) Delete
c) Search
d) Display tree (Traversal)
e) Display - Depth of tree
f) Display - Mirror image
g) Create a copy
h) Display all parent nodes with their child nodes
i) Display leaf nodes
j) Display tree level wise*/
```

```
#include<iostream>
using namespace std;
int i=0,leafCount=0;
struct node
{
int data;
node *left;
node *right;
};
class BST
{
node *root;
public:
node* Create(int arr[], int n)
{
node *newNode,*temp,*ptr;
for(int i=0;i<n;i++)
{
newNode=new node;
newNode->data=arr[i];
newNode->left=NULL;
newNode->right=NULL;
if(i==0)
{
root=newNode;
}
else
{
temp=root;
while(temp!=NULL)
{
ptr=temp;
if(newNode->data<temp->data)
{
temp=temp->left;
}
else
{
temp=temp->right;
}
}
if(newNode->data<ptr->data)
{
ptr->left=newNode;
}
else
{

```

```

ptr->right=newNode;
}
}
}
return root;
}
void Display1(node *root)
{
if(root!=NULL)
{
cout<<" "<<root->data;
Display1(root->left);
Display1(root->right);
}
}
void Display2(node *root)
{
if(root!=NULL)
{
Display2(root->left);
cout<<" "<<root->data;
Display2(root->right);
}
}
void Display3(node *root)
{
if(root!=NULL)
{
Display3(root->left);
Display3(root->right);
cout<<" "<<root->data;
}
}
int Search(node *root,int key)
{
if(root!=NULL)
{
if(root->data==key)
{
cout<<"\nData found!!! ";
i=1;
}
Search(root->left,key);
Search(root->right,key);
}
return i;
}
int TreeDepth(node *root)
{
int hl,hr;
if(root==NULL)
{
return 0;
}
if(root->left==NULL && root->right==NULL)
{
return 1;
}
hl=TreeDepth(root->left);
hr=TreeDepth(root->right);

```

```

return (max(hl,hr)+1);
}
int DisplayLeaf(node *root)
{
if(root==NULL)
{
return 0;
}
if(root->left==NULL && root->right==NULL)
{
cout<<" "<<root->data;
leafCount++;
}
DisplayLeaf(root->left);
DisplayLeaf(root->right);
return 0;
}
};
int main()
{
node *ROOT;
int n;
cout<<"Enter how many node do you want to create :";
cin>>n;
int arr[n];
bool a=true;
int temp;
for(int i=0;i<n;i++)
{
cin>>temp;
for(int j=0;j<i;j++)
{
if(arr[j]!=temp)
{
arr[i]=temp;
}
else
{
cout<<" Dupliicate Entry!!"<<endl;
return 0;
}
}
if(i==0)
{
arr[i]=temp;
}
}
BST bst;
int ch,b;

do
{
cout<<"\n1. Create Tree \n2. Display Tree \n3. Search Node \n4. Depth of
Tree \n5. Display Leaf Node \n6. Exit \n";
cout<<"Enter your choise : ";
cin>>ch;
if(ch==1)
{
ROOT=bst.Create(arr,n);
cout<<"\nBinary Tree Created Successfully!!! \n";

```

```

}
else if(ch==2)
{
cout<<"\nPre-order : \n";
bst.Display1(ROOT);
cout<<"\nIn-order : \n";
bst.Display2(ROOT);
cout<<"\nPost-order : \n";
bst.Display3(ROOT);
}
else if(ch==3)
{
int key;
cout<<"\nEnter key data for search : ";
cin>>key;
int b=bst.Search(ROOT,key);
}
else if(ch==4)
{
cout<<"\nHeight of Tree : "<<bst.TreeDepth(ROOT)<<endl;
}
else if(ch==5)
{
bst.DisplayLeaf(ROOT);
cout<<" \nLeaf Node Count : "<<leafCount;
if(b==0)
{
cout<<"\nData not found!!! ";
}
}
else
{
return 0;
}
cout<<"\nDo you want to Continue (1/0) :";
cin>>a;
}while(a);
}

```

/*OUTPUT:

Enter how many node do you want to create :4

66

92

4

21

1. Create Tree

2. Display Tree

3. Search Node

4. Depth of Tree

5. Display Leaf Node

6. Exit

Enter your choise : 1

Binary Tree Created Successfully!!!

Do you want to Continue (1/0) :1

1. Create Tree

2. Display Tree
3. Search Node
4. Depth of Tree
5. Display Leaf Node
6. Exit

Enter your choise : 2

Pre-order :

66 4 21 92

In-order :

4 21 66 92

Post-order :

21 4 92 66

Do you want to Continue (1/0) :1

1. Create Tree
2. Display Tree
3. Search Node
4. Depth of Tree
5. Display Leaf Node
6. Exit

Enter your choise : 3

Enter key data for search : 92

Data found!!!

Do you want to Continue (1/0) :1

1. Create Tree
2. Display Tree
3. Search Node
4. Depth of Tree
5. Display Leaf Node
6. Exit

Enter your choise : 4

Height of Tree : 3

Do you want to Continue (1/0) :1

1. Create Tree
2. Display Tree
3. Search Node
4. Depth of Tree
5. Display Leaf Node
6. Exit

Enter your choise : 5

21 92

Leaf Node Count : 2

Data not found!!!

Do you want to Continue (1/0) :1

1. Create Tree
2. Display Tree
3. Search Node
4. Depth of Tree
5. Display Leaf Node
6. Exit

Enter your choise : 6

```
...Program finished with exit code 0  
Press ENTER to exit console.*/  

```